LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE CR002
PERTEMUAN 6

Tema, Routing dan Navigasi, serta Animasi

Disusun Oleh:

Arum Syafitri Puspita Pratiwi (20240801010)

Dosen Pengampu:

JEFRY SUNUPURWA ASRI , S.Kom., M.Kom

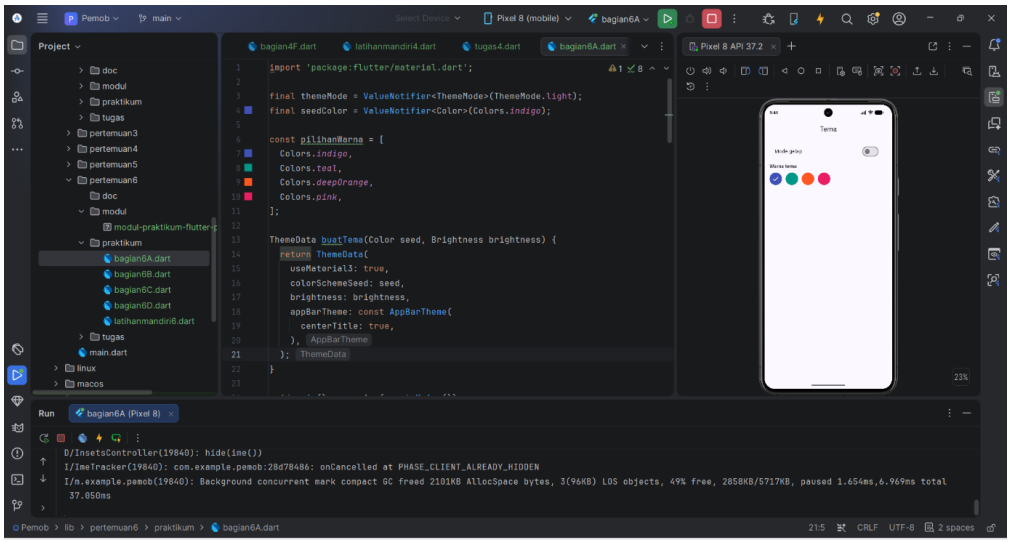
PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

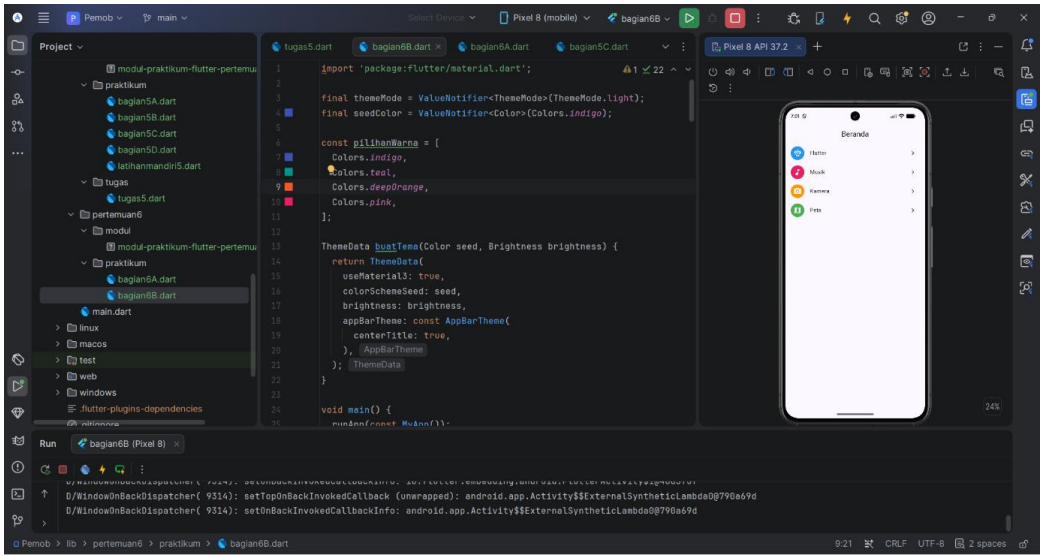
2026

Bagian A: Tema Terang/Gelap dan Warna Tema



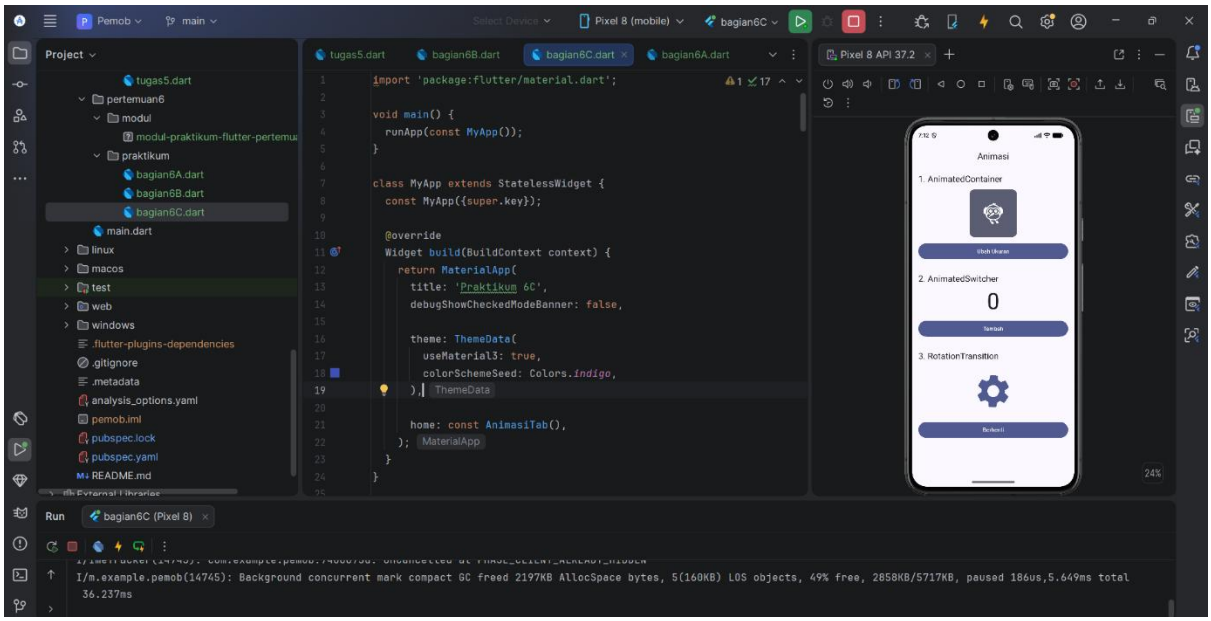
Pada praktikum Bagian 6A ini, dilakukan penerapan tema terang/gelap serta kustomisasi warna aplikasi menggunakan ValueNotifier dan ListenableBuilder pada lib/main.dart. Dari hasil *running* di emulator, antarmuka pengaturan berupa saklar mode gelap dan tombol pilihan warna berhasil berjalan dengan lancar. Tampilan aplikasi, termasuk AppBar, bisa langsung berubah secara *real-time* saat warna atau mode diganti tanpa perlu *restart* aplikasi. Hal ini terjadi karena seluruh elemen visual memanfaatkan tema bawaan Material 3 melalui ThemeData, sehingga warna aplikasi tidak perlu diatur manual pada tiap-tiap widget.

Bagian B: Named Routes, Argumen, dan Hero



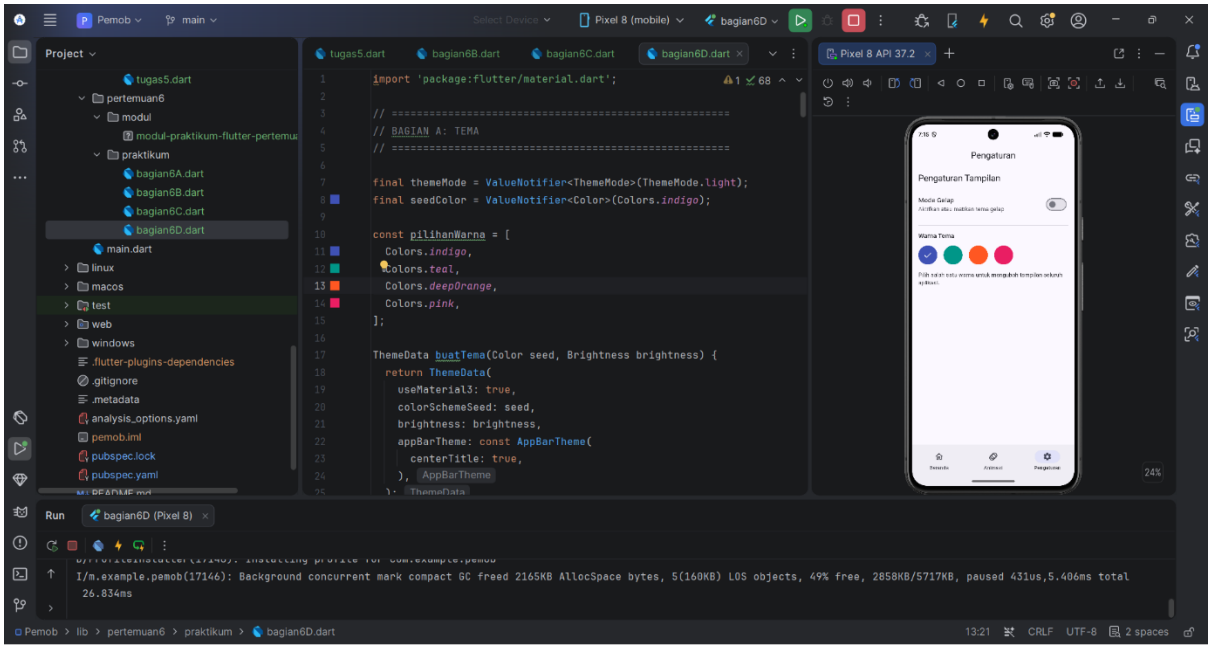
Pada praktikum Bagian 6B ini, fokus diarahkan pada penerapan sistem *named routes*, pengiriman argumen antar halaman, serta animasi transisi Hero. Berdasarkan hasil *running* pada emulator, aplikasi berhasil menampilkan daftar item pada halaman utama dengan ikon yang dilengkapi widget Hero. Ketika salah satu item ditekan, aplikasi mulus mengarahkan pengguna ke halaman detail melalui rute bernama, di mana ikon tersebut terbang dan membesar secara dinamis menuju posisi barunya serta menampilkan informasi detail sesuai item yang dipilih.

Bagian C: Animasi



Pada praktikum Bagian 6C ini, diimplementasikan beberapa jenis animasi di Flutter yang mencakup animasi implisit, transisi komponen, dan animasi eksplisit. Berdasarkan hasil *running* pada emulator, aplikasi berhasil menampilkan tiga demo animasi dengan lancar, yaitu perubahan ukuran dan warna pada AnimatedContainer, efek pergantian angka dengan transisi skala pada AnimatedSwitcher, serta ikon roda gigi yang berputar secara terus-menerus menggunakan AnimationController dan dapat dikontrol tombol interaktif.

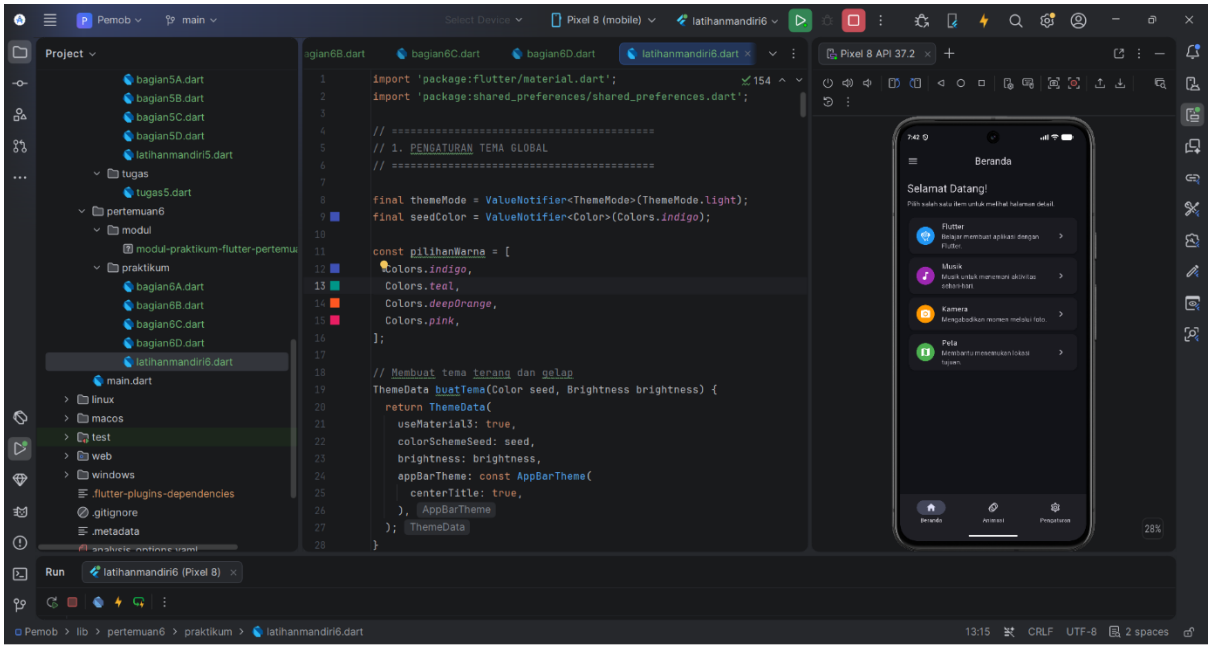
Bagian D: Navigasi Tab Bawah (Menyatukan Semuanya)



Pada praktikum Bagian 6D ini, seluruh komponen dari bagian sebelumnya disatukan ke dalam sebuah kerangka utama (ShellPage) menggunakan navigasi tab bawah (NavigationBar) dan widget IndexedStack untuk menjaga kondisi setiap halaman tetap aktif saat berpindah. Berdasarkan hasil *running* pada emulator, aplikasi berhasil menampilkan tiga menu tab utama

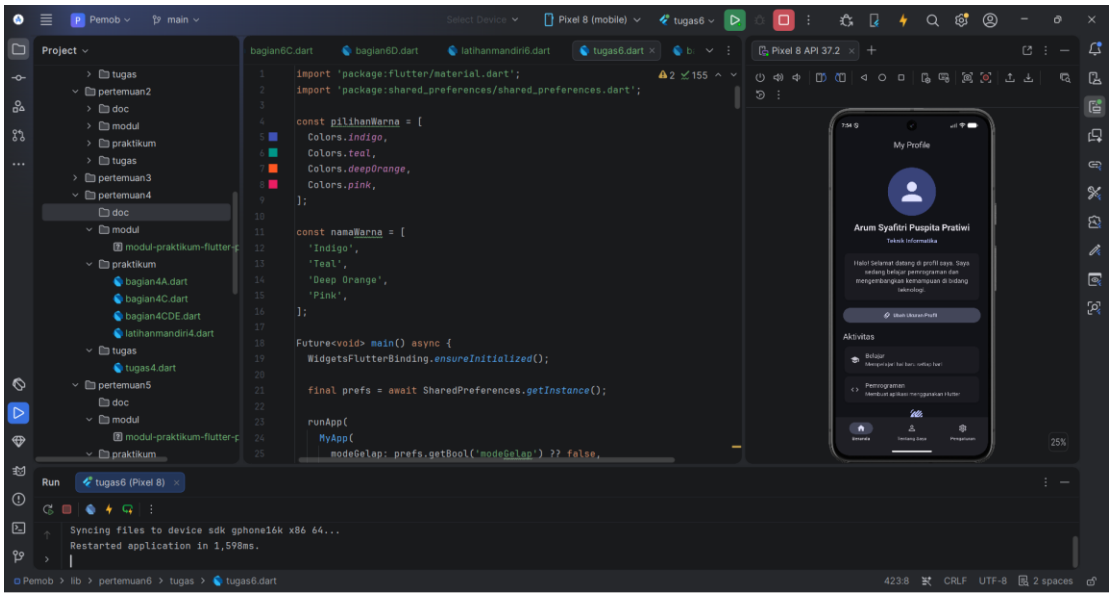
di bagian bawah, yaitu tab Beranda untuk navigasi item dengan animasi Hero, tab Animasi untuk menjalankan berbagai demo pergerakan, serta tab Pengaturan yang dapat mengubah tema terang/gelap dan palet warna ke seluruh bagian aplikasi secara *real-time*.

Latihan Mandiri



Pada bagian latihan mandiri ini, aplikasi dikembangkan lebih lanjut dengan mengintegrasikan penyimpanan lokal SharedPreferences agar pilihan mode gelap dan warna tema tetap tersimpan saat aplikasi ditutup. Selain itu, antarmuka utama diperkaya dengan penambahan menu Drawer di samping navigasi tab bawah, penerapan transisi FadeTransition pada perpindahan halaman detail, penanganan rute tak dikenal melalui halaman *404*, serta penambahan demo animasi keempat berupa AnimatedOpacity pada tab animasi untuk memudarkan dan memunculkan kartu secara interaktif.

Tugas



Pada tugas akhir praktikum ini, dibangun sebuah aplikasi profil diri interaktif yang dilengkapi dengan navigasi tiga tab menggunakan NavigationBar, sistem *named routes* untuk membuka halaman detail dengan animasi transisi Hero, serta penerapan animasi implisit (AnimatedContainer) dan eksplisit (AnimationController). Berdasarkan hasil *running* pada emulator, aplikasi juga telah berhasil menerapkan fitur pengaturan tema terang/gelap serta berbagai pilihan warna yang disimpan secara permanen menggunakan SharedPreferences, di mana seluruh elemen antarmuka dan warna telah disesuaikan secara konsisten menggunakan sistem tema Theme.of(context).

Pertanyaan Refleksi

1. Apa perbedaan animasi implisit dan eksplisit? Kapan memilih masing-masing?

   Jawab: Animasi implisit (seperti AnimatedContainer atau AnimatedOpacity) adalah animasi di mana Flutter yang mengurus proses pergerakannya secara otomatis cukup dengan mengubah nilai parameternya lewat setState. Sedangkan animasi eksplisit (menggunakan AnimationController) membutuhkan kontrol penuh dari pengembang untuk mengatur jalannya animasi secara manual (mulai, berhenti, mengulang, atau menentukan durasi secara spesifik).

   Kapan memilih: Pilih animasi implisit untuk kebutuhan transisi sederhana yang tidak rumit. Pilih animasi eksplisit saat membutuhkan kontrol penuh yang berkelanjutan atau kompleks (seperti animasi yang berputar terus-menerus atau dikontrol secara presisi)

2. Mengapa dua Hero harus memiliki tag yang sama, dan apa yang terjadi bila tag-nya kembali muncul dua kali di halaman yang sama?

   Jawab: Dua widget Hero harus memiliki tag yang sama agar Flutter tahu objek mana dari halaman asal yang harus dihubungkan dan ditransisikan menuju halaman tujuan. Jika terdapat tag yang sama muncul dua kali pada halaman yang sama, Flutter akan mengalami kebingungan (*duplicate tag error*) karena tidak dapat menentukan elemen mana yang dimaksud untuk dipasangkan.

3. Apa keuntungan memakai Theme.of(context) dibanding menulis warna langsung pada widget?

   Jawab: Menggunakan Theme.of(context) membuat aplikasi menjadi konsisten dan mendukung perubahan tema secara otomatis (seperti beralih antara mode terang dan mode gelap) tanpa perlu mengubah warna pada setiap widget satu per satu. Sebaliknya, menulis warna secara langsung (*hard-coded*) akan membuat warna tersebut kaku dan tidak bisa menyesuaikan saat tema aplikasi diganti.

4. Apa perbedaan berpindah tab dengan IndexedStack dan membuka halaman dengan Navigator?

   Jawab: Berpindah tab dengan IndexedStack hanya menyembunyikan atau menampilkan widget di dalam satu layar yang sama, sehingga status (*state*) dari setiap tab akan tetap tersimpan dan tidak hilang meskipun berpindah-pindah. Sementara itu,

membuka halaman dengan Navigator akan menumpuk halaman baru di atas halaman sebelumnya (*push*), yang biasanya akan membuat halaman lama tertutup atau terhapus dari memori tergantung pengaturannya.

5. Mengapa AnimationController wajib di-dispose?

   Jawab: AnimationController wajib di-*dispose* di dalam fungsi dispose() agar memori dan sumber daya sistem yang digunakan oleh pengendali animasi tersebut dapat dibersihkan sepenuhnya saat widget ditutup, sehingga terhindar dari masalah kebocoran memori (*memory leak*).

Kesimpulan

Berdasarkan praktikum Pertemuan 6, dapat disimpulkan bahwa Flutter menyediakan berbagai fitur untuk membuat aplikasi yang interaktif dan nyaman digunakan, mulai dari pengaturan tema terang dan gelap, navigasi menggunakan named routes dan NavigationBar, hingga penerapan animasi implisit, eksplisit, dan Hero. Penggunaan ThemeData dan Theme.of(context) membantu menjaga konsistensi tampilan aplikasi, sedangkan SharedPreferences memungkinkan pengaturan pengguna tetap tersimpan meskipun aplikasi ditutup. Melalui praktikum ini, pemahaman mengenai pengelolaan tema, perpindahan halaman, pemanfaatan animasi, serta penyusunan beberapa fitur dalam satu aplikasi menjadi lebih baik, sehingga dapat diterapkan dalam pengembangan aplikasi Flutter yang lebih terstruktur dan interaktif.